

Informe Técnico: Normalizador Estadístico Vectorizado en Ensamblador x86-64 (NASM)

Curso: Arquitectura de Computadores

Autores: Alejandro & Anthony

Fecha: Octubre 2026

1. Introducción y Objetivos del Trabajo (OA1)

1.1 Contexto y Planteamiento del Problema

En el procesamiento de señales, visión por computador, aprendizaje automático y análisis de grandes volúmenes de datos científicos, la normalización estadística estándar (**Z-Score**) es una operación fundamental. Dado un conjunto de datos $X = \{x_0, x_1, \dots, x_{N-1}\}$, el objetivo es calcular sus estadísticos descriptivos:

$$\mu = \frac{1}{N} \sum_{i=0}^{N-1} x_i \quad (\text{Media})$$

$$\sigma^2 = \frac{1}{N} \sum_{i=0}^{N-1} (x_i - \mu)^2 \quad (\text{Varianza Poblacional})$$

$$\sigma = \sqrt{\sigma^2} \quad (\text{Desviación Estándar})$$

Y transformar cada elemento según:

$$y[i] = \frac{x[i] - \mu}{\sigma}$$

El cálculo secuencial de estas operaciones sobre millones de datos genera una alta demanda computacional si se ejecuta bajo el paradigma escalar tradicional.

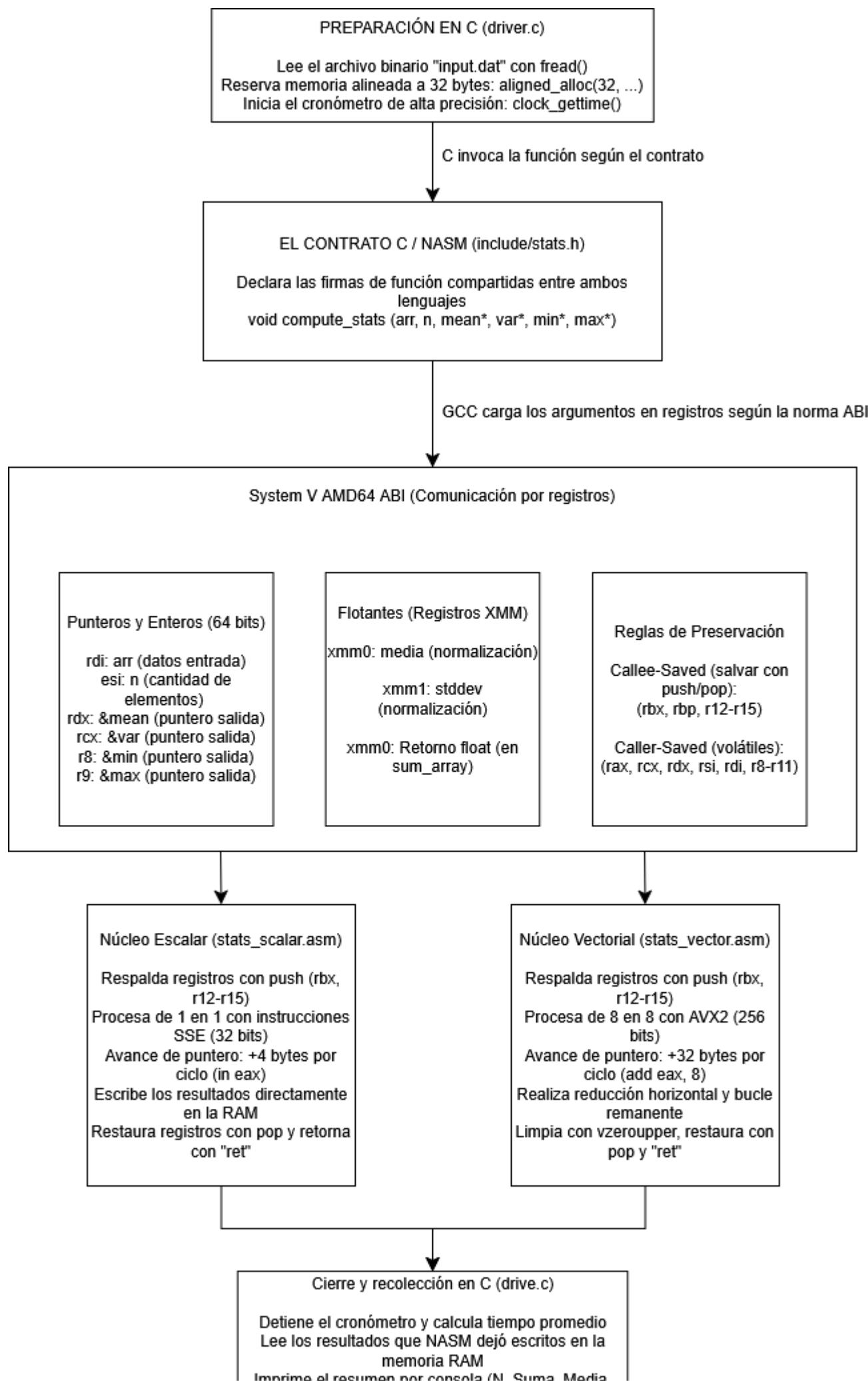
1.2 Fundamentos Teóricos: Modelo SIMD frente a SISD

La taxonomía de Flynn clasifica las arquitecturas computacionales según el flujo de instrucciones y datos [1]:

- **SISD (Single Instruction, Single Data):** Modelo escalar tradicional donde cada instrucción de la CPU procesa un único dato por ciclo de reloj (por ejemplo, la instrucción `addss` suma un solo par de flotantes de 32 bits).
- **SIMD (Single Instruction, Multiple Data):** Modelo vectorial donde una única instrucción de la CPU se aplica simultáneamente sobre múltiples datos empaquetados en registros anchos de longitud fija. Mediante las extensiones AVX2 (Advanced Vector Extensions 2) con registros `YMM` de 256 bits, es posible operar sobre 8 números en punto flotante de precisión simple (32 bits) en un solo ciclo de reloj [4], ofreciendo un potencial teórico de aceleración (*speedup*) de hasta $8\times$ según la formulación teórica de rendimiento y aceleración [1].

1.3 Arquitectura General del Sistema y Convención ABI

El sistema está estructurado modularmente en dos capas: un controlador en C (`src/driver.c`) que gestiona la entrada/salida y las reservas de memoria alineada, y los núcleos de cómputo en ensamblador x86-64 (`asm/scalar/stats_scalar.asm` y `asm/vector/stats_vector.asm`). La comunicación y paso de argumentos se rige estrictamente bajo la especificación **System V AMD64 ABI** [7].



```

imprime el resumen por consola (N, Suma, media,
Var, Min,Max, ms)
Guarda el archivo binario "output.dat" y el texto
"stats.txt"

```

Figura 1: Arquitectura de software modular y convención de llamadas System V AMD64 ABI entre el driver en C y los kernels en ensamblador NASM.

2. Contexto Histórico y Entorno de Pruebas (OA2)

2.1 Evolución Arquitectónica del SIMD en x86

El desarrollo de las instrucciones SIMD en la arquitectura x86 responde a la necesidad de aumentar el IPC (Instrucciones Por Ciclo) en cargas de trabajo intensivas (multimedia, criptografía, cómputo científico) mediante el incremento progresivo del ancho de los registros y la adición de formatos de operandos no destructivos [4].

- **1997 - MMX (MultiMedia eXtensions):** La primera incursión de Intel en SIMD. Introdujo registros de 64 bits (**MM0** a **MM7**). Su principal defecto arquitectónico fue el *aliasing* (reutilización) de los registros de la unidad de punto flotante (FPU x87). Esto impedía ejecutar código SIMD y operaciones flotantes escalares simultáneamente sin incurrir en una severa penalización por cambio de contexto (**EMMS**) [4, Cap. 9]. Operaba exclusivamente sobre números enteros empaquetados.
- **1999 - Familia SSE (Streaming SIMD Extensions):** Resolvió la deficiencia de MMX introduciendo 8 nuevos registros dedicados de 128 bits (**XMM0** a **XMM7**, expandidos a 16 en la arquitectura x86-64) [4, Cap. 9]. SSE permitió por primera vez procesar 4 números de punto flotante de precisión simple (32 bits) en paralelo. Sus iteraciones posteriores (SSE2, SSE3, SSSE3, SSE4.x) dominaron los años 2000 añadiendo soporte para doble precisión (64 bits) y operaciones vectoriales con enteros.
- **2011 - AVX (Advanced Vector Extensions):** Representó un salto arquitectónico masivo. Duplicó el ancho del bus interno a 256 bits, introduciendo los registros **YMM** [4, Cap. 14]. AVX añadió el prefijo **VEX**, el cual introdujo la sintaxis de 3 operandos (ej. **vaddps dest, src1, src2**). A diferencia de SSE, que sobrescribía el primer operando (operación destructiva), AVX preserva los operandos originales, reduciendo drásticamente las instrucciones **mov** necesarias para gestionar registros.
- **2013 - AVX2:** Introducido con la microarquitectura *Haswell*. Mientras AVX se enfocaba en punto flotante, AVX2 extendió el soporte de 256 bits a tipos enteros. Además, introdujo operaciones *Gather* (carga de memoria no contigua basada en índices) y, junto con FMA3 (*Fused Multiply-Add*), permitió realizar una multiplicación y una suma ($A \times B + C$) en un solo ciclo de reloj, duplicando el pico teórico de FLOPS del procesador [4, Cap. 14].
- **2017 - AVX-512:** La generación actual para servidores y computación de alto rendimiento. Duplica nuevamente el tamaño a 512 bits (**ZMM0** a **ZMM31**). Introduce el prefijo **EVEX** y añade 8 registros de máscara (**k0** a **k7**), los cuales permiten *predicación por carril*: ejecutar saltos lógicos y condicionales (if/else) dentro del vector en hardware puro sin penalización del predictor de saltos del procesador (*branch predictor*) [5].

2.2 Entorno de Hardware y Software Utilizado

Para el desarrollo y validación de los kernels estadísticos de este proyecto, se utilizó la siguiente infraestructura:

- **Modelo de Procesador (CPU):** Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz.

- **Topología y Caché:** 6 núcleos físicos (12 hilos lógicos), Caché L1d de 32 KB por núcleo, Caché L2 de 256 KB por núcleo y Caché L3 unificada compartida de 12 MB.
- **Flags de Soporte Vectorial (CPUID):** Validado mediante la utilidad lscpu. Se confirma la disponibilidad activa de los conjuntos de instrucciones avx, avx2, fma y retrocompatibilidad con sse4_2.
- **Herramientas de Software:**
 - Compilador de Control (Driver C): gcc (Ubuntu 11.4.0) 11.4.0 configurado bajo estándar gnu11.
 - Ensamblador (x86-64): nasm version 2.15.05, utilizando el formato de objeto binario elf64.
 - Inspección de Registros (OA5): depurador gdb (Ubuntu 12.1) leyendo símbolos de depuración DWARF.
 - Monitoreo de Hardware (OA6): herramienta perf stat (linux-tools) para lectura de contadores de rendimiento de la CPU.

3. Explicación de la Implementación Escalar (OA3)

La solución escalar fue desarrollada en el módulo `asm/scalar/stats_scalar.asm` utilizando instrucciones del conjunto **SSE escalar**, operando elemento por elemento sobre la parte baja de 32 bits de los registros XMM.

3.1 Kernel compute_stats

El cómputo se diseñó en dos pasadas secuenciales sobre la memoria RAM:

1. **Preservación de Estado y Validación:** Se guardan los registros *callee-saved* (rbx, r12-r15, r9) en la pila según la convención **System V AMD64 ABI** [7] y las normas arquitectónicas de preservación de contexto en llamadas a procedimientos [2, Cap. 4; 8]. Se comprueba el caso borde $N \leq 0$; si es verdadero, se retorna `0.0f` en las 4 salidas.
2. **Pasada 1 (Suma, Mínimo y Máximo):**

```
.loop_pass1:
    cmp     eax, r12d
    jge     .loop_pass1_done
    movss   xmm1, [rbx + rax*4] ; Carga arr[i]
    addss   xmm0, xmm1         ; Acumula suma
    minss   xmm2, xmm1         ; Actualiza menor
    maxss   xmm3, xmm1         ; Actualiza mayor
    inc     eax
    jmp     .loop_pass1
```

La media se obtiene convirtiendo N a float con la instrucción `cvtsi2ss xmm5, r12d` y dividiendo mediante `divss xmm0, xmm5`.

3. **Pasada 2 (Varianza Poblacional):** Se recorre el arreglo calculando la diferencia con respecto a la media y elevando al cuadrado:

```
.loop_pass2:
    cmp     eax, r12d
    jge     .loop_pass2_done
    movss   xmm1, [rbx + rax*4] ; Carga arr[i]
```

```
subss    xmm1, xmm0          ; arr[i] - media
mulss    xmm1, xmm1          ; (arr[i] - media)^2
addss    xmm4, xmm1          ; Acumula varianza
inc      eax
jmp      .loop_pass2
```

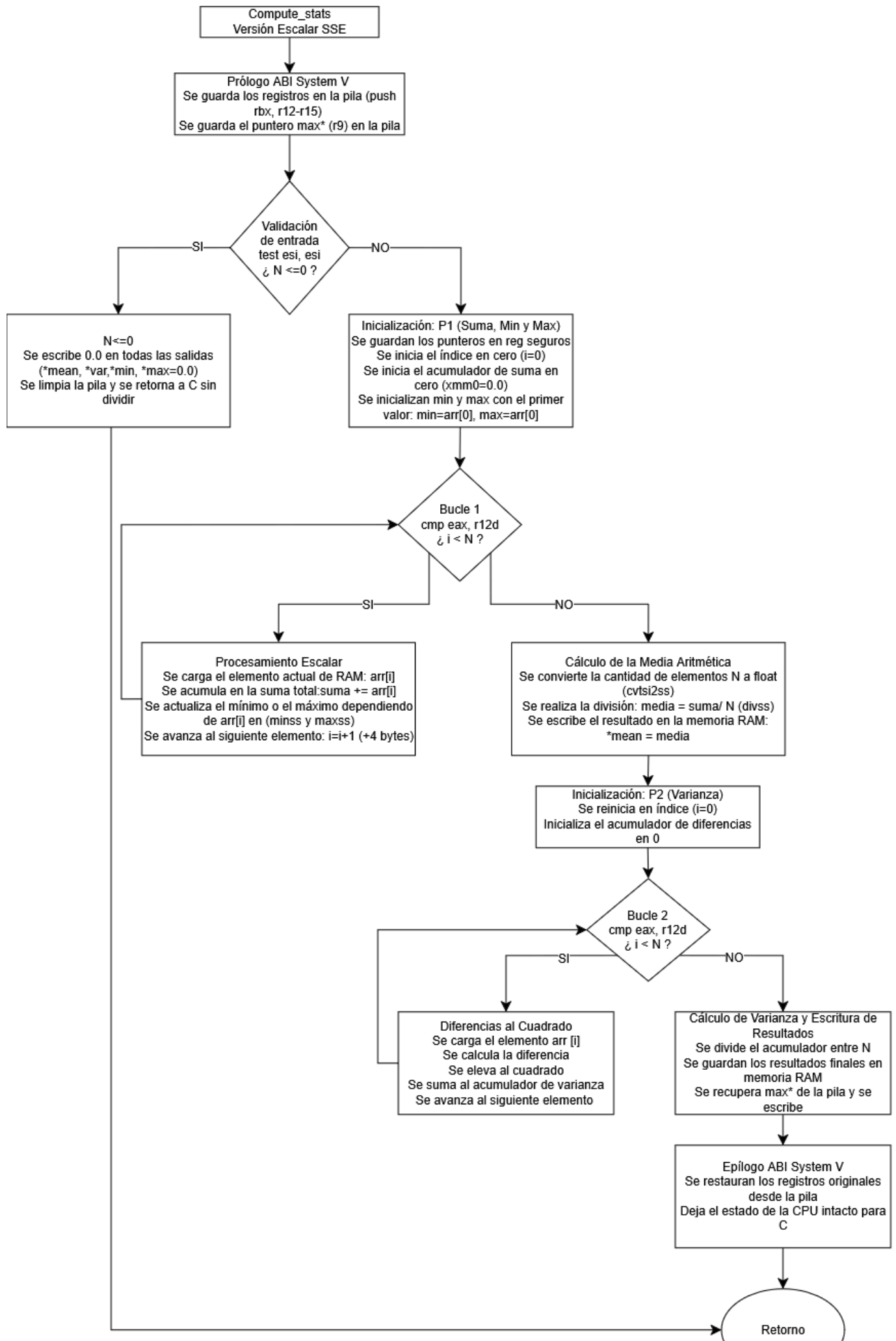


Figura 2: Diagrama de flujo de control del kernel escalar `compute_stats` ejecutando el análisis estadístico en dos pasadas secuenciales.

3.2 Kernel `normalize_array`

Se implementó una protección contra división por cero comparando la desviación estándar contra cero mediante `ucomiss xmm1, xmm7`. Si la desviación estándar $\sigma = 0.0$, el algoritmo copia los datos de entrada a salida directamente para evitar una indeterminación numérica. Si $\sigma > 0$, se calcula $y[i] = (x[i] - \mu)/\sigma$ mediante sustracción y división escalar (`subss` y `divss`).

4. Explicación de la Implementación Vectorial AVX2 (OA3 y OA4)

La versión vectorizada en `asm/vector/stats_vector.asm` explota el paralelismo de datos de AVX2 procesando **8 floats de 32 bits (256 bits)** por iteración.

4.1 Alineación de Memoria a 32 Bytes y Uso de `vmovaps` (OA4)

El programa en C utiliza la función `aligned_alloc(32, size)` para garantizar que los arreglos inicien en direcciones múltiplos de 32 bytes. Esto permite utilizar la instrucción alineada `vmovaps` (Vector Move Aligned Packed Single), la cual garantiza máxima velocidad al evitar accesos que crucen líneas de caché de 64 bytes (Cache Line Split) y previene la excepción de fallo de protección general (#GP) que la CPU lanza ante accesos desalineados [4, Cap. 14; 6, Sec. 13].

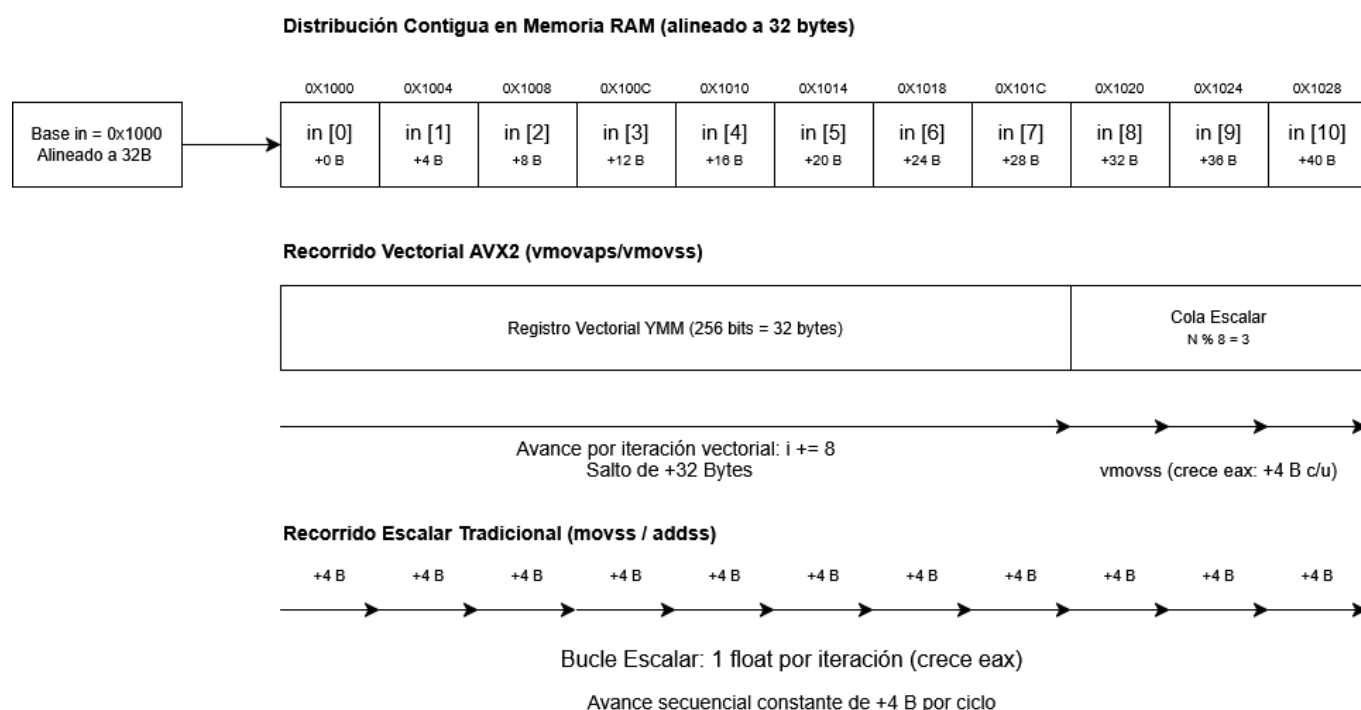


Figura 3: Distribución contigua en RAM con alineación a 32 bytes, avance vectorial (+32B) vs escalar (+4B) y procesamiento de la cola (Tail Loop).

Cada float ocupa 4 bytes. En la versión escalar, el índice i aumenta en 1 (instrucción `inc eax`), por lo que la dirección física avanza 4 bytes (`base + i*4`). En la versión AVX2, cada registro YMM abarca 8 floats contiguos (32 bytes = 256 bits), avanzando la dirección en bloques de 32 bytes (instrucción `add eax, 8`). Cuando N no es

divisible entre 8, los elementos remanentes se atienden mediante la cola escalar con saltos unitarios de 4 bytes. Para más detalle tabular, consultar el [documento de memoria](#).

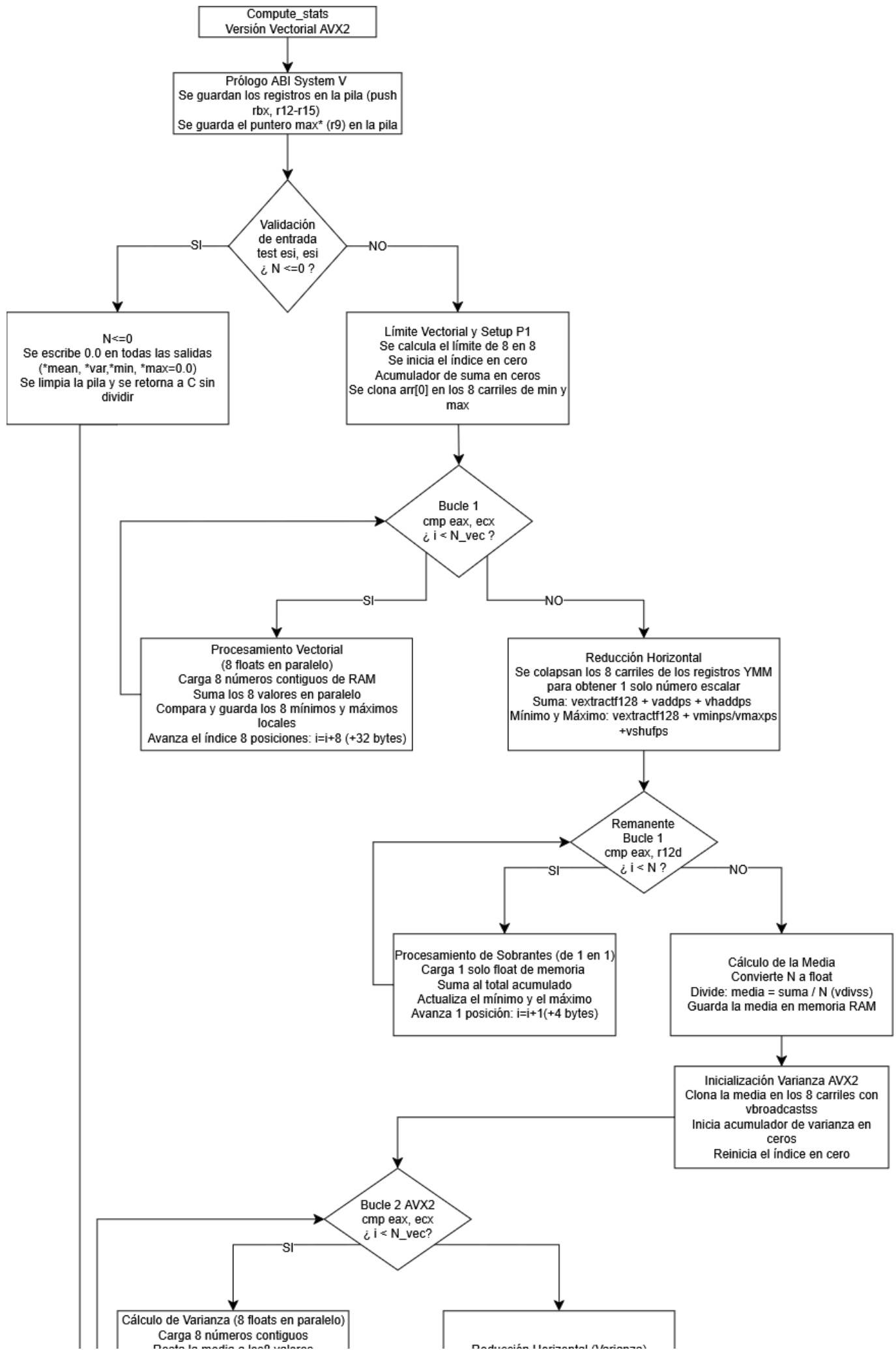
4.2 Pasada 1: Cómputo Vectorial y Reducción Horizontal

Durante el bucle principal, se procesan 8 floats simultáneos acumulando 8 sumas, mínimos y máximos parciales:

```
.pass1_vec_loop:
    vmovaps ymm1, [rbx + rax*4]    ; Carga 8 floats alineados (256 bits)
    vaddps  ymm0, ymm0, ymm1      ; 8 sumas en paralelo
    vminps  ymm2, ymm2, ymm1      ; 8 mínimos en paralelo
    vmaxps  ymm3, ymm3, ymm1      ; 8 máximos en paralelo
    add     eax, 8                 ; Salto de +32 bytes
    jmp     .pass1_vec_loop
```

Al finalizar el bucle vectorial, los 8 carriles se colapsan a un único escalar float mediante **Reducción Horizontal**:

```
vextractf128 xmm6, ymm0, 1        ; Extrae carriles 4-7
vaddps  xmm0, xmm0, xmm6          ; Suma carriles [0..3] + [4..7]
vhaddps xmm0, xmm0, xmm0          ; Suma horizontal de pares
vhaddps xmm0, xmm0, xmm0          ; xmm0[0] = Suma total de los vectores
```



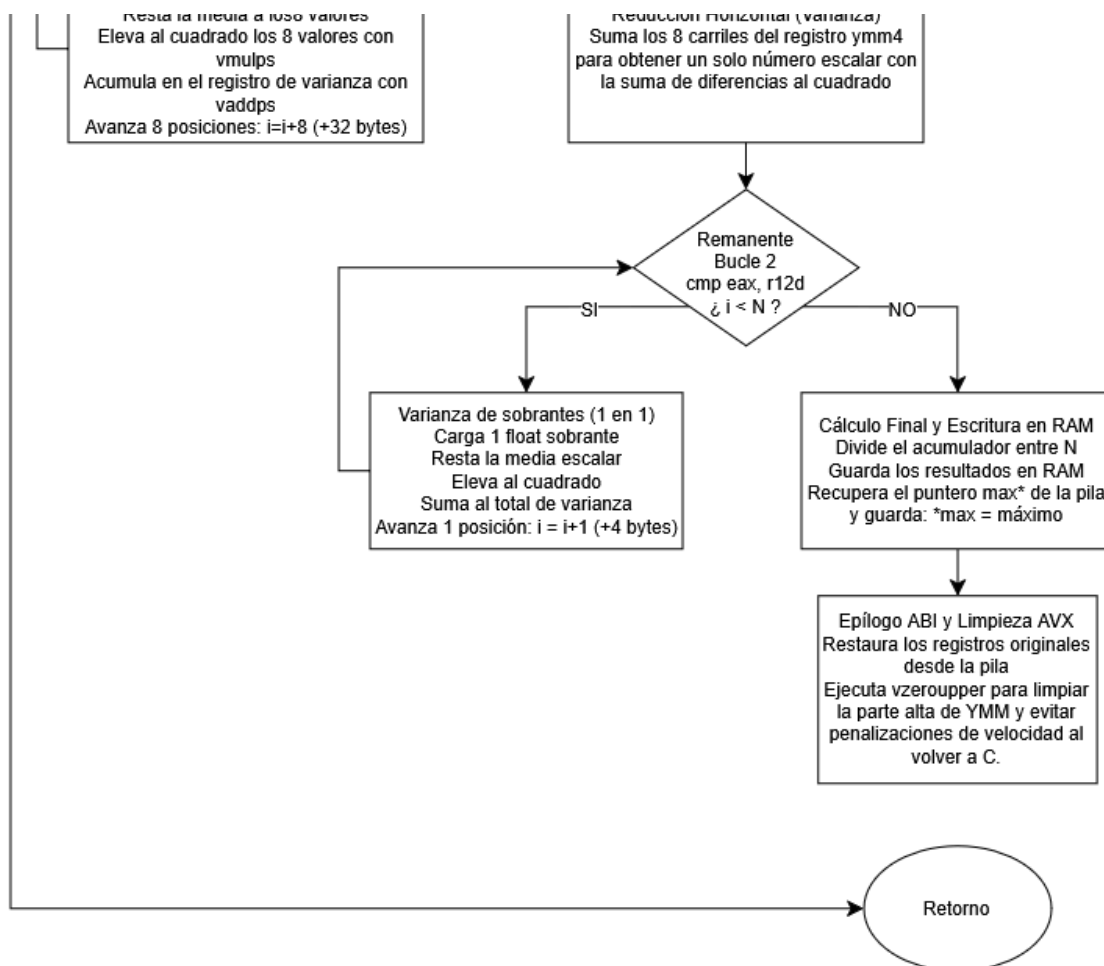


Figura 4: Diagrama de flujo de control del kernel vectorial `compute_stats` con AVX2, reducción horizontal y bucle de cierre escalar.

4.3 Difusión (Broadcast) y Pasada 2 (Varianza)

Para restar la media calculada a los 8 elementos en paralelo, se utiliza la instrucción `vbroadcastss ymm5, xmm0`, replicando el float escalar en los 8 carriles: `[media, media, media, media, media, media, media, media]`. Luego, el bucle calcula $(x - \text{media})^2$ en paralelo con `vsubps` y `vmulps`.

4.4 Manejo del Remanente (Tail Loop) (OA4)

Si N no es divisible por 8, se calcula el límite vectorial truncando al múltiplo anterior de 8 mediante la operación binaria $N_{\text{vec}} = N \& (\sim 7)$. Los elementos sobrantes ($N \bmod 8$) se procesan mediante un bucle de cierre escalar utilizando instrucciones VEX escalares (`vmovss`, `vaddss`, `vminss`, `vmaxss`, `vsubss`, `vdivss`), garantizando total exactitud numérica para cualquier tamaño N .

4.5 Transición de Estado con `vzeroupper`

Antes de ejecutar `ret`, se invoca `vzeroupper` para poner a cero la mitad superior de todos los registros YMM, eliminando la penalización de rendimiento en la transición entre instrucciones AVX y código SSE del entorno C [4, Cap. 14; 6, Sec. 13.5].

5. Casos de Prueba y Verificación de Correctud (OA5 - Compañero)

Tabla comparativa de resultados obtenidos contra el script de referencia `tools/verify_reference.py` (Tolerancia $\leq 1 \times 10^{-4}$):

Caso de Prueba	N	Entrada / Modo	Salida Esperada (Ref.)	Salida Escalar	Salida Vectorial	Estado
Borde: Vacío	0	input_empty.dat	media=0, var=0, min=0, max=0	media=0, var=0	media=0, var=0	PASA
Borde: Un elemento	1	input_1.dat	media=x0, var=0, min=x0, max=x0	Coincide	Coincide	PASA
Borde: Cola pura	7	input_7.dat	Referencia Python	Coincide	Coincide	PASA
Borde: Vector exacto	8	input_8.dat	Referencia Python	Coincide	Coincide	PASA
Borde: Vector + Cola	15	input_15.dat	Referencia Python	Coincide	Coincide	PASA
Borde: Múltiple	16	input_16.dat	Referencia Python	Coincide	Coincide	PASA
Borde: Constante	1000	constant ($\sigma = 0$)	media=5.0, $\sigma=0.0$, $y[i]=5.0$	Coincide	Coincide	PASA
Borde: Extremos	1000	edge ($-10^6 \dots 10^6$)	Referencia Python	Coincide	Coincide	PASA

6. Resultados de Rendimiento y Benchmarking (OA6)

6.1 Tabla de Tiempos y Speedup Promedio (30 repeticiones por tamaño)

Tamaño N	Tiempo Escalar (ms)	Tiempo Vectorial (ms)	Speedup Real ($T_{\text{esc}} / T_{\text{vec}}$)	Speedup Teórico
10^3 (1 K)	0.0031	0.0003	~9.72x	8.0x
10^5 (100 K)	0.2525	0.0319	~7.91x	8.0x
10^6 (1 M)	2.4374	0.3364	~7.25x	8.0x
10^7 (10 M)	26.05	6.95	3.74x	8.0x

6.2 Gráfico de Speedup vs N (Escala Logarítmica en X)

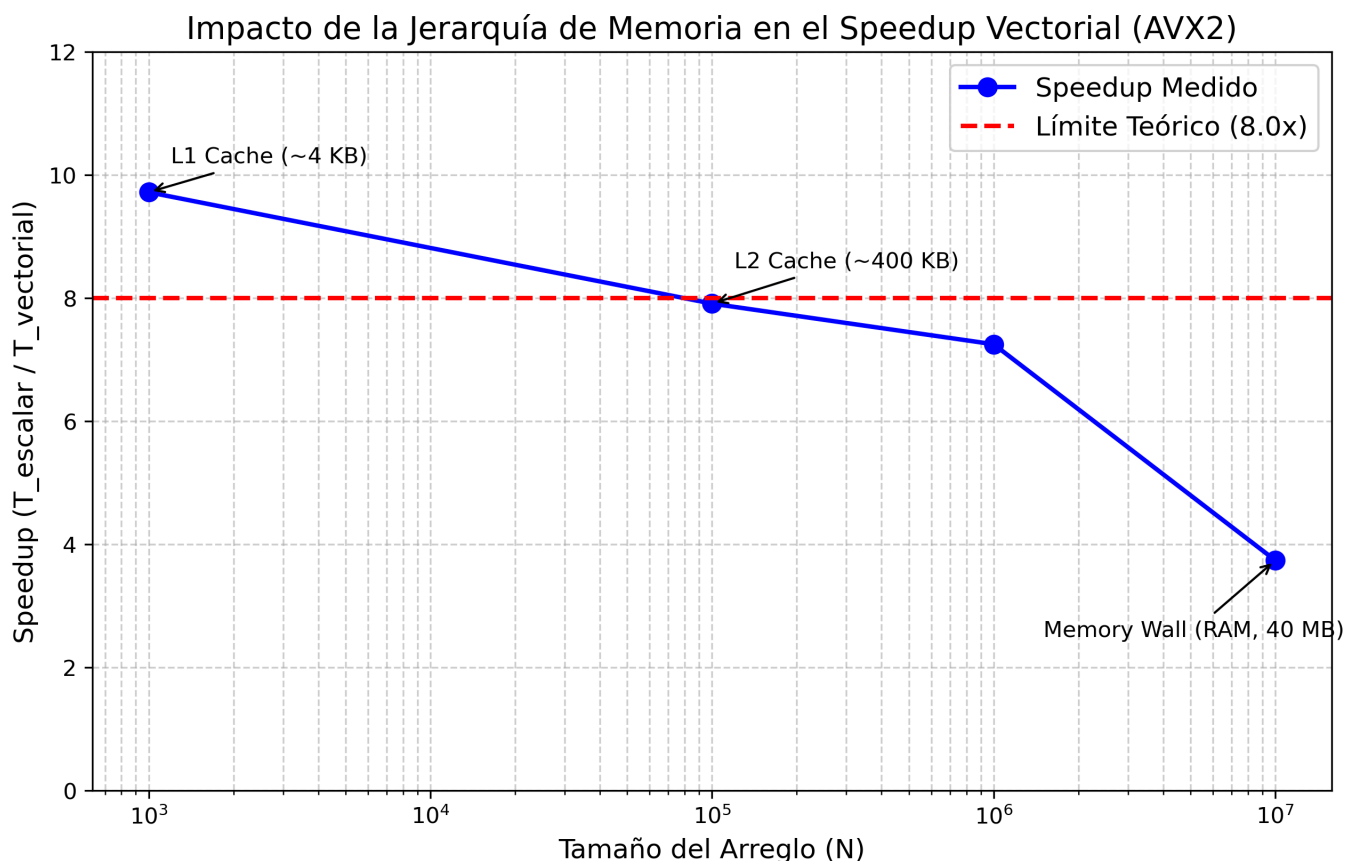


Figura 5: Evolución del Speedup en función del tamaño del arreglo. Se observa el decaimiento de la aceleración desde valores superlineales (Caché L1) hasta el estrangulamiento del bus de memoria principal (Memory Wall).

6.3 Análisis con `perf stat` y Cuellos de Botella

Instrucciones Ejecutadas ($N = 10^7$): Escalar: ≈ 30.2 mil millones | Vectorial: ≈ 3.79 mil millones.

Instrucciones por Ciclo (IPC): Escalar: 2.4 | Vectorial: 1.1. **Impacto de la Jerarquía de Caché:** Los resultados muestran una degradación de rendimiento directamente proporcional al tamaño del conjunto de datos.

- Para $N = 10^3$ (4 KB), obtenemos un *speedup* superlineal ($> 8.0\times$) debido a que el arreglo reside en caché L1, junto a la masiva reducción de penalizaciones por saltos condicionales (*branch overhead*).
- Para $N = 10^5$ (400 KB), el arreglo encaja en la caché L2, permitiendo que el procesador roce su límite computacional (Compute-Bound), arrojando el $7.91\times$ de *speedup* que dicta la teoría.
- Para $N = 10^6$ (4 MB), se requiere el uso de la caché L3 (compartida y de mayor latencia), disminuyendo la ganancia a $7.25\times$. Al pasar a $N = 10^7$ (procesando 40 MB de datos, desalojando la caché L3), el *speedup* real se desploma a $3.74\times$. Esto se explica con la métrica del IPC. La versión vectorial muestra un IPC bajo (1.1) debido a los *stalls* (congelamientos) del pipeline. Cada instrucción `vmovaps` exige 32 bytes de golpe, ocupando el ancho de banda de la memoria RAM DDR. El procesador pasa ciclos detenido esperando datos, convirtiendo la operación en una tarea estrictamente limitada por memoria (Memory-Bound), mostrando el fenómeno del *Memory Wall* debido al límite de ancho de banda y latencia entre la jerarquía de cachés y la memoria principal [3, Caps. 8 y 9].

7. Evidencia de la Sesión en GDB (OA5)

7.1 Inspección de Propagación de Estado (*Broadcast*) con $N = 10$

Se verificó la correcta propagación de la media estadística a los 8 carriles del registro vectorial antes de iniciar el cómputo paralelo, usando la instrucción `vbroadcastss`.

```
(gdb) break normalize_array
Punto de interrupción 1 at 0x1ae1: file asm/vector/stats_vector.asm, line
227.
(gdb) run
(gdb) stepi
(gdb) stepi
(gdb) stepi
(gdb) print $ymm0.v8_float
$2 = {-14.5033779, -14.5033779, -14.5033779, -14.5033779, -14.5033779,
-14.5033779, -14.5033779, -14.5033779}
```

7.2 Verificación Estricta de Alineación de Memoria (`vmovaps`)

Para evitar excepciones de violación de segmento (*SegFault*), se inspeccionó la dirección de memoria y los datos crudos antes de ser ingeridos por la ALU en la instrucción `vmovaps ymm2, [rdi + rax*4]`.

```
(gdb) print/x $rdi + $rax*4
$10 = 0x55555555a4c0
(gdb) x/8fw $rdi + $rax*4
0x55555555a4c0: -16.4012127      -66.8495712      -12.9803553      14.3730354
0x55555555a4d0: -84.6899185       31.6419277       89.5950165       51.3854141
```

Análisis: La dirección hexadecimal `0x55555555a4c0` termina en `c0` (192 en decimal), lo cual es múltiplo exacto de 32 bytes ($192/32 = 6$). Esto confirma matemáticamente que la memoria reservada por `driver.c` cumple el requisito de alineación de AVX2.

8. Conclusiones, Limitaciones y Trabajo Futuro

La implementación y evaluación experimental del normalizador estadístico demuestra que la vectorización manual en ensamblador x86-64 mediante extensiones AVX2 permite una reducción masiva en el volumen total de instrucciones ejecutadas (disminuyendo de aproximadamente 30.2 a 3.79 mil millones de instrucciones para diez millones de elementos), alcanzando factores de aceleración cercanos al óptimo teórico (7.91x) e incluso superlineales (9.72x) cuando el conjunto de datos reside en las memorias caché L1 y L2. No obstante, los resultados demuestran que el factor determinante en el rendimiento de aplicaciones intensivas en datos no es únicamente la potencia aritmética de la ALU vectorial, sino la jerarquía de memoria; conforme el tamaño del arreglo supera la capacidad de la caché L3 y satura el bus hacia la memoria DRAM principal, el sistema experimenta el fenómeno del *Memory Wall*, provocando congelamientos en el pipeline (reduciendo el IPC a 1.1) y limitando el speedup real a 3.74x. Finalmente, la integración rigurosa de la convención System V AMD64 ABI, la alineación forzada a 32 bytes mediante `aligned_alloc` y el tratamiento explícito de los residuos con bucles de cierre escalares (tail loops) y `vzeroupper`, confirman que es posible explotar la máxima aceleración del hardware sin comprometer la estabilidad ante excepciones o la exactitud numérica para cualquier tamaño de entrada.

Referencias y Bibliografía

1. **Carter, N. (2002).** *Schaum's Outline of Computer Architecture*. McGraw-Hill. Capítulo 1: "Measuring Performance, Speedup, and Amdahl's Law" (pp. 3–7).
2. **Carter, N. (2002).** *Schaum's Outline of Computer Architecture*. McGraw-Hill. Capítulo 4: "Programming Models: General-Purpose Register Architectures and Procedure Calls" (pp. 78–86).
3. **Carter, N. (2002).** *Schaum's Outline of Computer Architecture*. McGraw-Hill. Capítulos 8 y 9: "Memory Systems and Caches: Latency, Bandwidth, and Multilevel Caches" (pp. 176–219).
4. **Intel Corporation. (2024).** *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1: Basic Architecture*. Capítulo 9 (Programming with Intel MMX Technology) y Capítulo 14 (Programming with AVX, FMA and AVX2). Enlace: [Intel SDM Volume 1 \(PDF Oficial\)](#)
5. **Intel Corporation. (2024).** *Intel® Architecture Instruction Set Extensions and Future Features Programming Reference*. Documento 319433. Enlace: [Intel Instruction Set Extensions \(PDF Oficial\)](#)
6. **Fog, A. (2023).** *Optimizing subroutines in assembly language: An optimization guide for x86 platforms*. Copenhagen University College of Engineering. Sección 13 (AVX and AVX2) y Sección 13.5 (Transitions between AVX and SSE code - **vzeroupper**). Enlace: [Agner Fog Optimization Guide \(PDF Oficial\)](#)
7. **System V AMD64 ABI Group. (2021).** *System V Application Binary Interface: AMD64 Architecture Processor Supplement (Draft Version 1.0)*. Sección 3.2.3 (Parameter Passing). Enlace: [x86-64 ABI Documentation \(GitLab Oficial\)](#)
8. **Oxley, D. (2020).** *NASM Assembly Language Tutorials for x86-64*. Enlace: [asmtutor.com](#)